Apache Spark: A Unified Engine for Big Data Processing

The rapid growth of data in industry and research creates major opportunities but also significant computational challenges. As single machines became insufficient, systems had to scale out across multiple nodes, leading to many new cluster programming models.

- Early systems were highly specialized (e.g., MapReduce for batch processing, Dremel for SQL queries, Pregel for graph algorithms).
- Open-source tools like Storm and Impala followed the same specialized trend. However, most big data applications require multiple processing types together: batch loading, SQL queries, and iterative machine learning.
- Using many separate engines increases complexity and reduces efficiency. Some applications cannot be expressed efficiently in any single specialized system.

What is Apache Spark and Why Was It Developed?

- The necessity to manage the diversity and irregularity of Big Data has given rise to many data processing projects, with Apache Spark aiming to solve this crucial problem.
- The project was initiated in 2009 by a working group at the University of California, Berkeley, with the goal of designing a unified engine for distributed data processing. While Spark employs a programming model similar to MapReduce, it extends this by introducing a data-sharing abstraction known as **Resilient Distributed Datasets (RDDs)**.

Key Advantages of Apache Spark and Comparative Analysis with Predecessor Systems

- Applications are easier to develop due to the utilization of a single, unified API.
- Combining processing tasks is significantly more efficient. While predecessor systems required writing data back to storage to transfer it to another engine, Spark can execute diverse functions on the same data, often in memory.
- Spark facilitates novel applications that were not feasible with previous systems, such as interactive queries on a graph and streamlined machine learning.
- Since its publication in 2010, Spark has emerged as one of the most active open-source projects in the field of big data processing. The project is currently utilized by over 1,000 organizations, spanning a diverse range of sectors, including technology companies, banking, retail, biotechnology, and astronomy.

Model Programming in Spark

- Spark’s core abstraction is the RDD (Resilient Distributed Dataset).
- RDDs are fault-tolerant, distributed collections of objects that can be processed in parallel.
- Users create RDDs using transformations such as map, filter, and groupBy.
- Spark provides a functional API in Scala, Java, Python, and R, allowing users to pass local functions that run across the cluster.

- For example, the following Scala code demonstrates the creation of an RDD that represents error messages within a log file by searching for lines starting with "ERROR." It then prints the total count of these errors.

```
lines = spark.textFile("hdfs://...")
errors = lines.filter(
  s => s.startsWith("ERROR"))
println("Total errors: "+errors.count())
```

Lazy Evaluation and Optimization in Spark

Spark evaluates RDDs lazily, allowing it to find an efficient plan for the user's computation. For ex ample, if there were multiple filter or map operations in a row, Spark can fuse them into one pass, or, if it knows that data is partitioned, it can avoid moving it over the network for groupBy. Users can thus build up programs modularly without losing performance.

Data Sharing and Persistence in Spark

RDDs provide explicit support for data sharing between computations.

RDDs are "ephemeral," meaning they are recomputed every time they are used in an action.

Users can explicitly persist selected RDDs in memory for fast reuse.

If the data does not fit in memory, Spark will also spill/write it to disk.

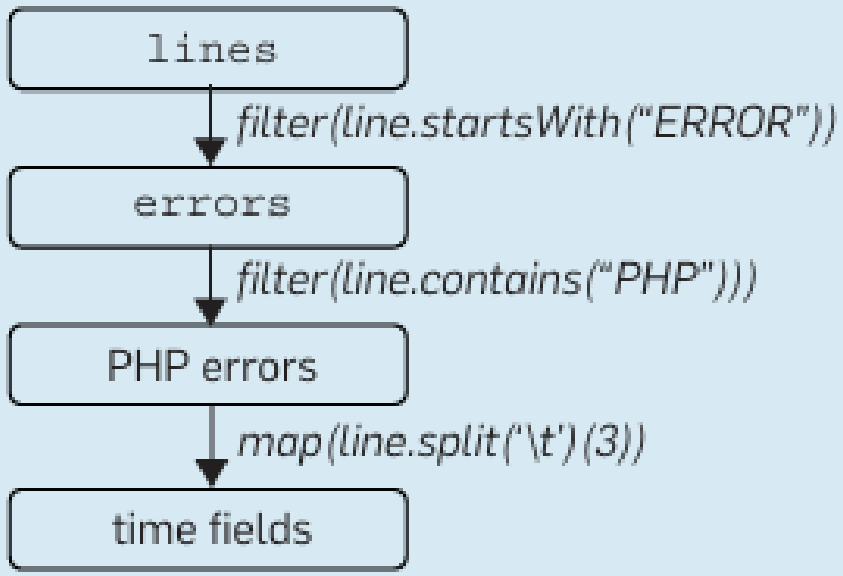
Fault Tolerance in Spark

In addition to enabling data sharing and various parallel operations, RDDs also possess the inherent ability to automatically recover from failures.

Traditionally, distributed computing systems achieved fault tolerance through either **replication** or **checkpointing**. Spark, however, employs a distinct approach based on what is termed **"lineage".**

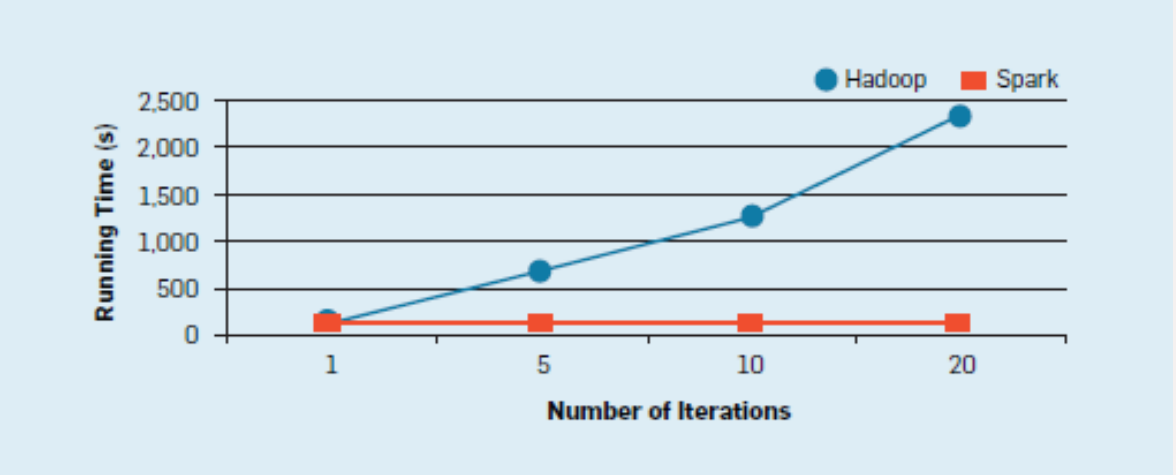
- Each RDD tracks the graph of transformations that was used to build it and reruns these operations on base data to reconstruct any lost partitions.
- If any partition of an RDD is lost, Spark will rebuild it by applying the filter on the corresponding block of the HDFS file.

Figure 2. Lineage graph for the third query in our example; boxes represent RDDs, and arrows represent transformations.



Efficiency of Lineage-Based Recovery

- Lineage-based recovery is significantly more efficient than replication in data-intensive workloads. It saves both time, because writing data over the network is much slower than writing it to RAM, and storage space in memory.
- Spark facilitates loading the data into RAM once, followed by the execution of multiple computations. Consequently, it operates significantly faster than traditional MapReduce.
- As demonstrated in the accompanying figure (on the right), for a 100 GB workload, MapReduce takes 110 seconds per iteration because it loads the data from disk in every iteration. In contrast, after the initial load, Spark requires only one second per subsequent iteration.



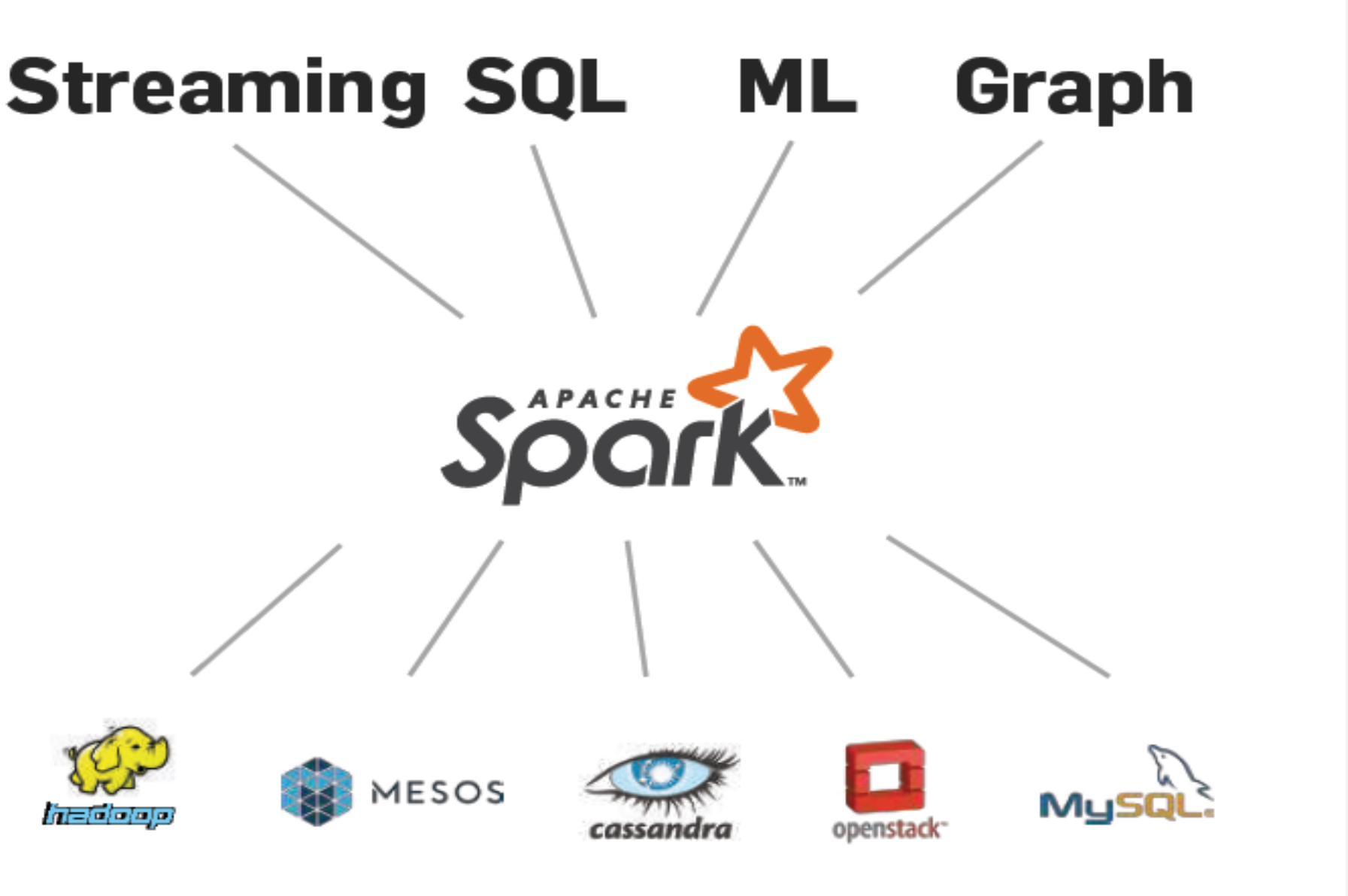
Integration With Storage Systems

Much like Google's MapReduce, Spark is designed to be used with multiple external systems for persistent storage. Spark is most commonly used with cluster file systems like HDFS and key-value stores like S3 and Cassandra.

RDDs usually store only temporary data within an application, though some applications also share RDDs across multiple users. Spark's design as a storage-system-agnostic engine makes it easy for users to run computations against existing data and join diverse data sources.

Higher-Level Libraries

- The RDD programming model provides only distributed collections of objects and functions to run on them. Using RDDs, however, have builted a variety of higher-level libraries on Spark, targeting many of the use cases of specialized computing engines.
- We now discuss the four main libraries included with Apache Spark.


Streaming
SQL
ML
Graph
APACHE
Spark
hadoop
MESOS
cassandra
openstack
MySQL

SQL and DataFrames

- Spark SQL (and predecessor Shark) implements relational queries using analytical database techniques.
- Supports columnar storage, cost-based optimization, and code generation.
- Uses compressed columnar storage layout directly inside RDDs.
- Records hold rows in binary format, and the system generates code to run directly against this layout.

- Beyond running SQL queries, we have used the Spark SQL engine to provide a higher-level abstraction for basic data transformations called DataFrames, which are RDDs of records with a known schema.
- DataFrames are a common abstraction for tabular data in R and Python, with programmatic methods for filtering, computing new columns, and aggregation.

Spark Streaming & GraphX

Spark Streaming

•Implements incremental processing via "discretized streams".

• Splits input into small batches (e.g., 200ms) combined with RDD state.

•Benefits:

- •Lower cost fault recovery due to lineage.
- •Combines streaming with batch and interactive queries.

GraphX

Provides graph computation similar to Pregel and GraphLab.

•Implements placement optimizations (e.g., vertex partitioning)using RDD partitioning functions.

MLlib

•MLlib,14 Spark’s machine learning library, implements more than 50 common algorithms for dis tributed model training.

•For example, it includes the common distributed algorithms of decision trees (PLANET), La tent Dirichlet Allocation, and Alternating Least Squares matrix factorization.

Combining Processing Tasks

- Spark's libraries all operate on RDDs as the data abstraction, making them easy to combine in applications.
- Figure 5 shows a program that reads some historical Twitter data using Spark SQL, trains a K-means clustering model using MLlib, and then applies the model to a new stream of tweets

Figure 5. Example combining the SQL, machine learning, and streaming libraries in Spark.

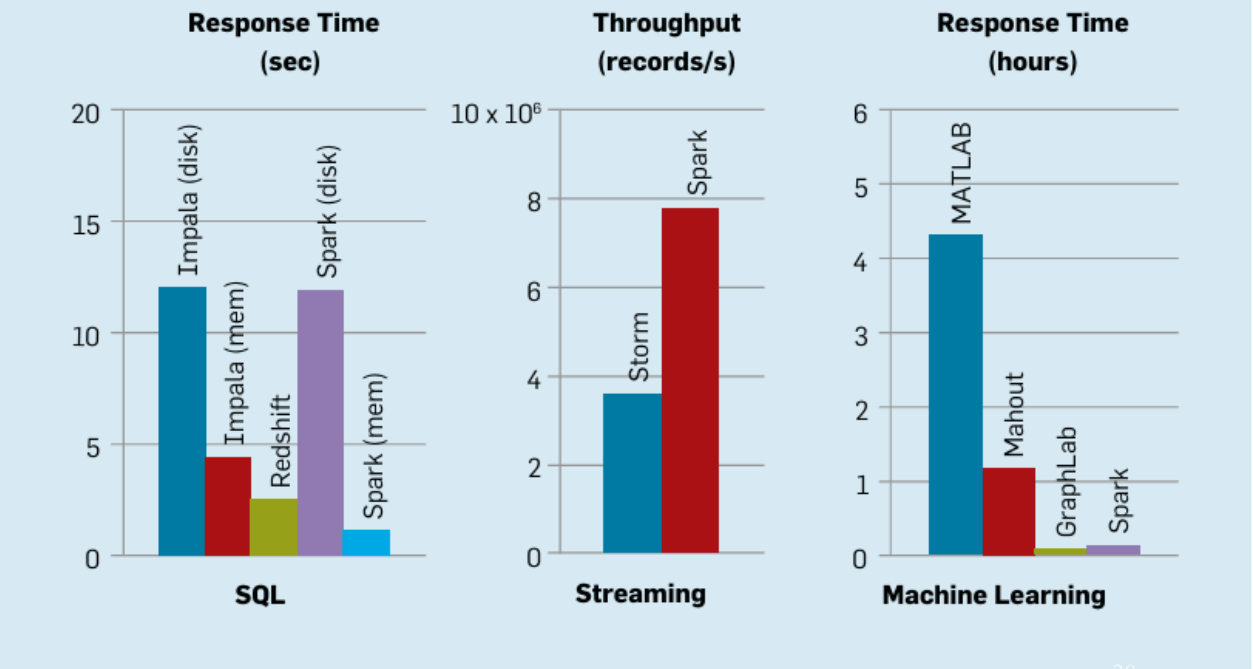
```
// Load historical data as an RDD using Spark SQL
val trainingData = sql(
   "SELECT location, language FROM old_tweets")

// Train a K-means model using MLlib
val model = new KMeans()
   .setFeaturesCol("location")
   .setPredictionCol("language")
   .fit(trainingData)
// Apply the model to new tweets in a stream
TwitterUtils.createStream(...)
      .map(tweet => model.predict(tweet.location))
```

Performance

- Figure 6 com pares Spark's performance on three simple tasks—a SQL query, streaming word count, and Alternating Least Squares matrix factorization—versus other engines. While the results vary across workloads, Spark is generally comparable with specialized systems like Storm, GraphLab, and Impala.

Figure 6. Comparing Spark's performance with several widely used specialized systems for SQL, streaming, and machine learning. Data is from Zaharia[24] (SQL query and streaming word count) and Sparks et al.[17] (alternating least squares matrix factorization).



Batch Processing

- Batch processing is Spark's most common application, covering ETL workloads and offline machine learning model training. Key examples include personalization at Yahoo!, data lake management at Goldman Sachs, and the largest published use case: Tencent's 8,000-node cluster ingesting 1PB of data per day. While many applications run on disk, Spark improves performance over MapReduce in these cases due to its ability to support more complex operator graphs.

Interactive Queries

Interactive use of Spark falls into three main classes.

- Organizations use Spark SQL for relational queries, often through businessintelligence tools like Tableau. Examples include eBay and Baidu.
- Developers and data scientists can use Spark's Scala, Python, and R interfaces interactively through shells or visual notebook environments. Such interactive use is crucial for asking more advanced questions and for designing models that eventually lead to production applications and is common in all deployments.
- Several vendors have developed domain-specific interactive applications that run on Spark. Examples include Tresata (anti-money laundering), Trifacta (data cleaning), and PanTera (largescale visualization)

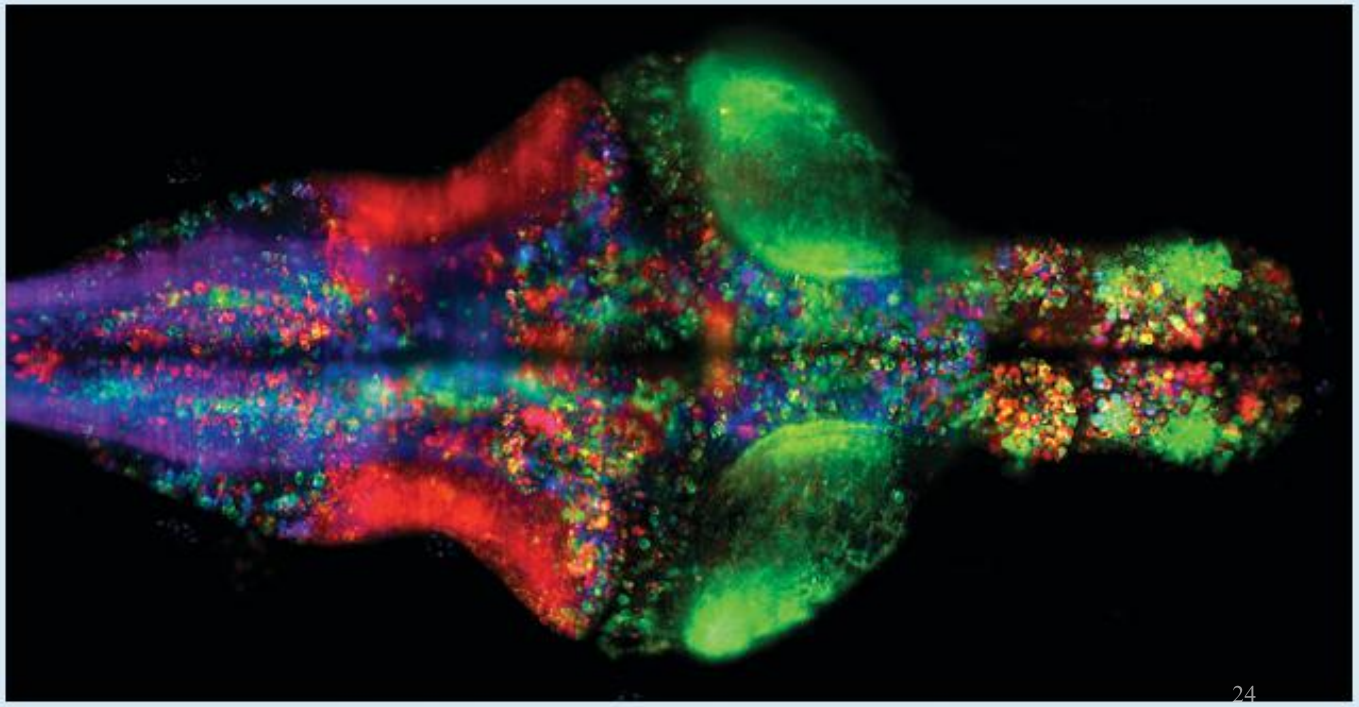
Stream Processing

- Real-time processing is also a popular use case, both in analytics and in real-time decisionstreaming making applications. Published use cases for Spark Streaming include network security monitoring at Cisco, prescriptive analytics at Samsung SDS, and log mining at Netflix. Many of these applications also combine streaming with batch and interactive queries.

Scientific Applications

- Spark is utilized in several scientific domains, including spam detection, image processing, and genomic data processing

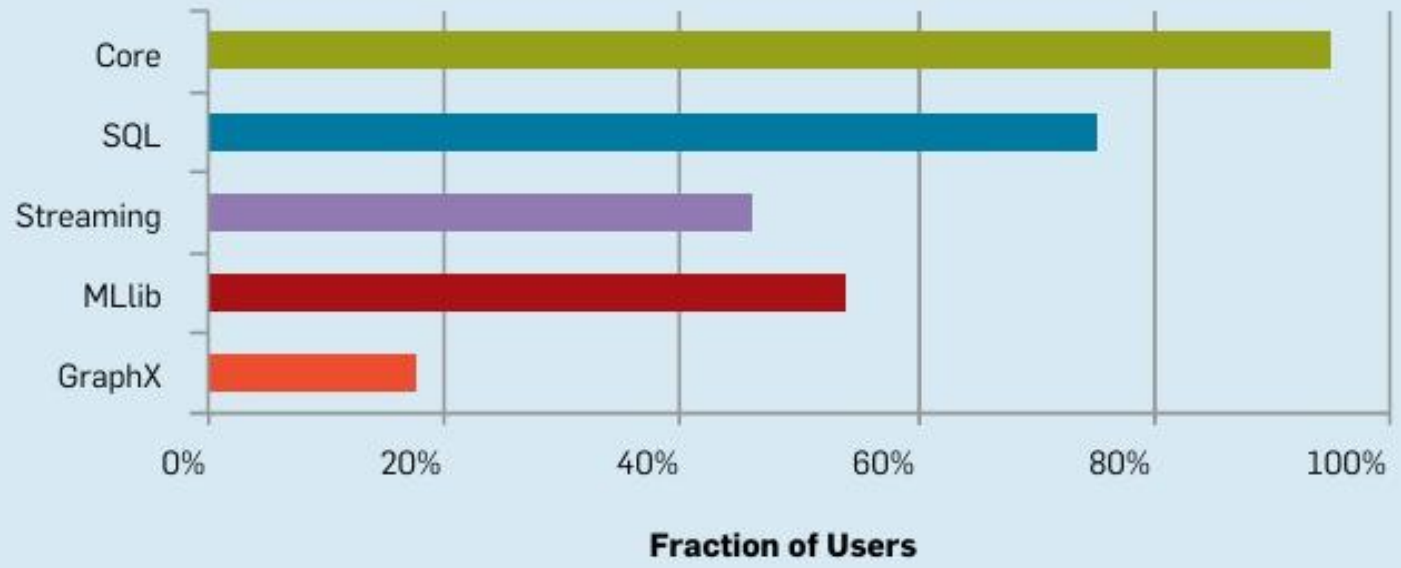
Figure 8. Visualization of neurons in the zebrafish brain created with Spark, where each neuron is colored based on the direction of movement that correlates with its activity. Source: Jeremy Freeman and Misha Ahrens of Janelia Research Campus.



Spark Components Used

- Spark is confirmed to be a unified engine, with surveys showing that over 60% of organizations use at least three of its APIs. Spark Core and SQL are the most popular components, but the majority use multiple components; 88% use at least two, and 60% use at least three.

Figure 9. Percent of organizations using each Spark component, from the Databricks 2015 Spark survey; https://databricks.com/blog/2015/09/24/.



Why Is the Spark Model General?

While Apache Spark has successfully demonstrated that a unified cluster programming model is both feasible and useful, it is beneficial to understand the factors that make cluster programming models generic, as well as Spark's inherent limitations.

In this context, we examine the generality of RDDs from two main perspectives:

- Expressiveness Perspective:

RDDs can emulate any distributed computation. As long as the computation is not sensitive to network latency, RDDs will perform this emulation efficiently in most cases.

- Systems Perspective

RDDs empower applications with control over the most common bottleneck resources in the cluster—network and storage I/O. This control makes it possible to express the same optimizations for these resources that characterize specialized systems.

Ongoing Work

- Apache Spark continues to evolve rapidly, with active contributions from both industry and academia.
- Since June 2013, the codebase has grown sixfold, largely due to the addition of new libraries.
- The ecosystem now includes over 200 third-party packages.
- In the research community, numerous projects at Berkeley, MIT, and Stanford are built on Spark.
- Many new libraries such as GraphX and Spark Streaming originated from research groups.
- This section provides an overview of four major ongoing efforts.

DataFrames and Declarative APIs

- Early Spark relied on RDDs, which were flexible but difficult to analyze and optimize due to arbitrary object structures and user-defined functions.
- In 2015, Spark introduced DataFrames, a declarative, table-based API inspired by relational algebra.
- DataFrames support optimized operations (filtering, aggregation) and allow developers to write code in languages like Python, R, Scala, and Java.

Figure 11. Example of Spark's DataFrame API in Python. Unlike Spark's core API, DataFrames have a schema with named columns (such as age and city) and take expressions in a limited language (such as `age > 20`) instead of arbitrary Python functions.

```
users.where(users["age"] > 20)
        .groupBy("city")
        .agg(avg("age"), max("income"))
```

Conclusion

- Scalable data processing is critical for the next generation of applications but usually requires complex multi-system workflows.
- Apache Spark addresses this by providing a unified engine and programming model for big data workloads.
- Experience shows that this unified approach delivers strong performance and significant usability benefits.
- Spark demonstrates the value of composable and interoperable libraries in large-scale data systems.

- -Dilara GÜVEN - 333078029
  - -Eren ATLI - 333078028